Problem Set 5

> In the completion of **Problem 1**, **Problem 4**, and the **Bonus Problem**, external assistance was utilized.
> Apart from this, *ChatGPT* was only used for language refinement and formatting throughout the assignment.
> No other part of the content was generated by AI.

Problem 1

Basic Idea

Create a new array that contains all distinct elements from the original array in sorted order. The length of this array is $O(\log n)$.

To construct this array, each element in the original array is searched within the current distinct array using binary search, which takes $O(n \log \log n)$ time in total. As new elements are inserted, some items in the distinct array need to be shifted, contributing an additional cost of $O((\log n)^2)$.

Thus, the total time complexity for this step is:

$$O(n \log \log n + (\log n)^2) = O(n \log \log n)$$

Next, create a `cnt` array of the same length as the distinct array. Scan through the original array again to count the occurrences of each distinct element. This process also requires a binary search for each element, resulting in a time cost of $O(n \log \log n)$.

Finally, reconstruct the complete sorted array by combining each element in the distinct array with its corresponding count in the `cnt` array.

pseudocode

```
Sort(array a):
    distinctArray = []
    for item in a:
```

```
        if item is not in distinctArray: # search cost O(loglogn) tim
e
            distinctArray.add(a) # add so that distinctArray is still
sorted, this is done O(logn) times, and each time costs O(logn) time
    cntArray = [0, 0,..., 0] # len same to distinct Array
    for item in a:
        if item is in distinctArray:# search cost O(loglogn) time
            cntArray[indexof item in distinctArray]++
    resultArray = []
    for item in distinctArray:
        for cntArray[indexof item in distinctArray]:
            resultArray.pushBack(item)
    return resultArray
```

Problem 2

Basic Idea

Use 27 buckets. For each string `s`, if `s[0]` is empty, put it in `bucket[0]`;
if `s[0] = 'a'`, put it in `bucket[1]`;
if `s[0] = 'b'`, put it in `bucket[2]`;
...
if `s[0] = 'z'`, put it in `bucket[26]`.

Leave `bucket[0]` as it is. For each of the other 26 buckets, create 27 subbuckets and place elements into these subbuckets according to their second character `s[1]`. If a bucket is empty or contains only one element, do nothing and create no subbuckets for it.

Repeat this process recursively until no bucket contains more than one element. Finally, extracting all items in the order of the buckets will produce the sorted list.

pseudocode

```
SortStrings(array strings, layer = 0):
    if strings.len == 0 or strings.len == 1:
        return strings
    buckets = bucket[27]
    for s in strings:
        if s.len == layer:
            bucket[0].addback(s)
            continue
```

```
            if s[layer] == 'a':
                bucket[1].addback(s)
            if s[layer] == 'b':
                bucket[2].addback(s)
            # 26branch cases
            # ...
            # ...
            if s[layer] == 'z':
                bucket[26].addback(s)
        for b in buckets:
            if b.len > 1:
                SortStrings(b, layer + 1)
        result = []
        for b in buckets:
            result += b
        return result
```

Time Complexity

For each string `s` with k characters, it is guaranteed to be placed into (or taken out of) a bucket $k+1$ times, which costs $O(k)$ time. Summing over all strings, the total cost is $O(n)$.

The longest string has at most n characters, so the number of layers is $O(n)$.
Consequently, the total number of buckets is also $O(n)$, and the cost of creating and destroying these buckets is $O(n)$.

Therefore, the total time complexity of the algorithm is:

$$O(n)$$

Problem 3

This data structure can be applied to any objects that are comparable.
Therefore, using it to implement **HEAPSORT** would make HEAPSORT a comparison-based sorting algorithm.

If both `INSERT` and `EXTRACTMAX` operations could be performed in $O(1)$ time,
then HEAPSORT would run in $O(n)$ time overall.
However, this contradicts the known lower bound for comparison-based sorting algorithms, which is $\Omega(n \log n)$.

Thus, such a data structure cannot exist.

Problem 4

(a)

The decision tree for this pairing problem has $n!$ leaves.
Assuming the tree is ternary, its depth is $\Theta(\log(n!)) = \Theta(n \log n)$.

Using the adversary argument, for a leaf at the deepest level,
$\Theta(n \log n)$ tests must be performed; otherwise,
there would exist an instance where the algorithm produces an incorrect result.

Therefore, at least $\Omega(n \log n)$ tests are required.

(b)

Assume there exists an algorithm that can pair $\frac{n}{2}$ objects, and that this algorithm requires $o(n \log n)$ tests.
Then, the total number of tests can be expressed as:

$$T(n) = o(n \log n) + T\left(\frac{n}{2}\right) = o(n \log n)$$

This contradicts the conclusion in part (a), where the entire problem was proven to have a lower bound of $\Omega(n \log n)$ tests.
Therefore, such an algorithm cannot exist.

Bonus Problem

First, we provide an **upper bound** using the following algorithm:

Use a tournament tree to find the largest element.
This requires $n - 1$ comparisons.
Then, consider all the elements that have been compared with the largest element and lost.
There are a total of $\lceil \log_2 n \rceil$ such elements, and the **second largest** must be the largest among them.
Finding this element requires $\lceil \log_2 n \rceil - 1$ additional comparisons.

Therefore, the total number of comparisons in the worst case is:

$$n - 1 + (\lceil \log_2 n \rceil - 1) = n - 2 + \lceil \log_2 n \rceil$$

Next, we prove the **lower bound** using an adversary argument:

The second largest element must have been compared with the largest one; otherwise, it would be impossible to determine whether it is the largest or the second largest.

Assume that the largest element has been compared m times.
Thus, there are m candidates for the second largest element.
Finding the largest takes $n-1$ comparisons,
and finding the second largest among these m candidates requires $m-1$ additional comparisons.
In total, this gives:

$$n-1+(m-1)=n+m-2$$

Now, we show that $m \geq \log_2 n$.

In the adversary argument, assign each element a **weight** equal to the number of elements it is greater than or equal to.
In each comparison, the element with the higher weight wins.
The winner's weight increases by at most a factor of two.

Therefore, for any element to become the overall largest, it must have participated in at least $\lceil \log_2 n \rceil$ comparisons.
Hence, $m = \lceil \log_2 n \rceil$, and the **lower bound** is:

$$n-2+\lceil \log_2 n \rceil$$

Problem 5

In the i-th iteration, $i-1$ items are already sorted.
When inserting a new item into this sorted sequence, it may need to be compared and moved past anywhere from 0 to $i-1$ elements, each with equal probability.

Thus, the expected number of items moved in this iteration is:

$$E(X_i) = \frac{0+1+2+\cdots+(i-1)}{i} = \frac{(i-1)i/2}{i} = \frac{i-1}{2}$$

By the **linearity of expectation**, the total expected number of item comparisons and moves is:

$$\sum_{i=1}^{n} \frac{i-1}{2} = \Theta(n^2)$$

Therefore, the expected time complexity of the algorithm is $\Theta(n^2)$.

Problem 6

(a)

Perform **MERGESORT** on the array of elements, then list out the i largest elements. The total time cost of this approach is:

$$O(n \log n)$$

pseudocode

```
Combine(array left, array right):
    lp = rp = 0
    result = []
    while lp < left.len and rp < right.len:
        if left[lp] <= right[rp]:
            result.addback(left[lp])
            lp++
            continue
        else:
            result.addback(right[rp])
            rp++
    while lp < left.len:
        result.addback(left[lp])
        lp++
    while rp < right.len:
        result.addback(right[rp])
        rp++
    return result

MergeSort(array a):
    if a.len == 1 or a.len == 0:
        return a
    left = MergeSort(a[0 to a.len/2 - 1])
    right = MergeSort(a[a.len/2 to a.len - 1])
    return Combine(left, right)

GetILargest(array a):
    MergeSort(a)
    result = []
    from 1 to i:
        result.addback(a.popback())
```

```
    return result
```

(b)

Build a heap from bottom to top in $O(n)$ time by performing a **sink** operation on each element.
Then, call `ExtractMax` i times to obtain the i largest elements.
Each extraction takes $O(\log n)$ time, so the total cost is:

$$O(n + i \log n)$$

pseudocode

```
Heapify(i, array a):
    n = a.len
    if i * 2 + 1 >= n:
        return
    if i * 2 + 2 < n and a[i * 2 + 2] > a[i * 2 + 1] and a[i * 2 + 2]
> a[i]:
        swap(a[i], a[i * 2 + 2])
        Heapify(i * 2 + 2, a)
        return
    if a[i * 2 + 1] > a[i]:
        swap(a[i], a[i * 2 + 1])
        Heapify(i * 2 + 1, a)
        return


BuildHeap(array a):
    for i from a.len-1 to 0:
        Heapify(i, a)

ExtractMax(array a):
    result = a[0]
    a[0] = a.popBack(a)
    Heapify(0, a)
    return result

GetLargest(array a):
    BuildHeap(a)
    result = []
    for 1 to i:
        result.addBack(ExtractMax(a))
```

```
    return result
```

(c)

Use **QuickSelect** with a **median-of-medians** pivot selection, which guarantees a worst-case runtime of $O(n)$.
Partitioning around the selected element also takes $O(n)$ time.
Finally, sorting the i largest elements requires $O(i \log i)$ time.

Therefore, the total running time is:

$$O(n + i \log i)$$

```
MedianOfMedian(array a):
    G1, G2, ..Gn/5 = makeGroups(a) # devide a into n/5 groups
    for i form 1 to n/5:
        MergeSort(Gi)
    medians = MedianElements(G1, G2,...Gn/5)
    return QuickSelect(medians, (n/5)/2)

QuickSelect(array a, i):
    if a.len == 1:
        return a[0]
    piviot = MedianOfMedian(a)
    array left
    array right
    for ele in a:
        if ele >= piviot:
            right.pushBack(ele)
        else:
            left.pushBack(ele)
    if i <= right.len:
        return QuickSelect(right, i)
    else:
        return QuickSelect(left, i - right.len)

MergeSort(array a) # same to (a)

GetLargest(array a):
    piviot = QuickSelect(a, i)
    result = []
    for ele in a:
        if ele >= piviot:
            result.pushBack(ele)
    MergeSort(result)
```

```
        return result
```